Video-Coder: Reinforcing Agentic Coding for Video Generation

Chaoyang Wang[1] **Sicheng Gao**[2] **Junan Hu**[3] **Kunyang Li**[1] **Dongyang Chen**[4]
Yiyang Zhou[5] **Huaxiu Yao**[5] **Yong Jae Lee**[6] **Radu Timofte**[2] **Yuzhang Shang**[1†]

[1] University of Central Florida [2] University of Würzburg [3] Microsoft Research
[4] Tsinghua University [5] UNC-Chapel Hill [6] UW–Madison

Home: https://github.com/CYWang735/Video-Coder
HF: https://huggingface.co/Video-Coder

ABSTRACT

Recent years have witnessed remarkable progress in video generation in terms of visual quality and temporal consistency. However, generating videos with precise spatial structure, coherent motion dynamics, and controllable scene composition remains challenging. In this paper, we introduce Video-Coder, the first attempt to train a code-driven video generation agent, which autonomously writes and executes code to construct a dynamic 3D scene (Draft Video) that provides structure and motion signals to downstream video generators. To support this paradigm, we design a dedicated data pipeline and curate two high-quality datasets, Video-Coder-SFT-6k and Video-Coder-RL-2k, which contain diverse prompts and ground-truth reference videos. We further develop the Video-Coder Harness, a unified interactive environment integrating code execution, 3D scene rendering, and execution feedback for agent training and inference. We then train Video-Coder with SFT followed by harness-native agentic reinforcement learning with dual reward feedback, which combines execution-based and video-based rewards. Additionally, we design an annealing training strategy that gradually reduces the execution reward over time, allowing the model to rely more on the video reward in the later training stages. Experiments show that Video-Coder achieves significant gains, improving Minimax-H3 by approximately 7 points on PhyGenBench and 24 points on VBench-2.0. We hope this work can serve as an open foundation for coding agents in video generation, and we fully open-source our data, models, and code.

1 Introduction

Video generation has advanced rapidly with diffusion-based and autoregressive models [1, 2, 3, 4, 5], achieving remarkable visual fidelity and short-term temporal consistency. However, these models learn to fit the video distribution in pixel or latent space without explicitly modeling the underlying 3D world. When prompts involve precise spatial relationships, multi-object interactions, or physically governed motion, the generated videos often exhibit incorrect object counts and positions, inconsistent occlusions, implausible trajectories, and objects that deform or vanish [6, 7, 8, 9, 10]. Moreover, end-to-end generated videos are hard to edit or reproduce, leaving users to rewrite prompts by trial and error.

Meanwhile, large language models (LLMs) have shown strong capabilities in code generation and agentic tasks [11, 12, 13]. Code is a precise, executable, and composable representation that naturally describes scene geometry, object motion, and camera trajectories: a short program can unambiguously specify that "three spheres fall simultaneously from different heights", a constraint that pixel-level models struggle with. Prior work has used LLMs to generate code for 3D scene construction or visual content creation [14, 15, 16, 17, 18], but mostly in a single-turn manner or with fixed prompting pipelines, without refining code from execution feedback or optimizing for final video quality.

Motivated by these observations, we propose Video-Coder, a code-driven video generation agent that formulates video generation as an agentic programming task. As shown in Figure 1, given a text prompt, the agent writes and executes

[†]Corresponding author

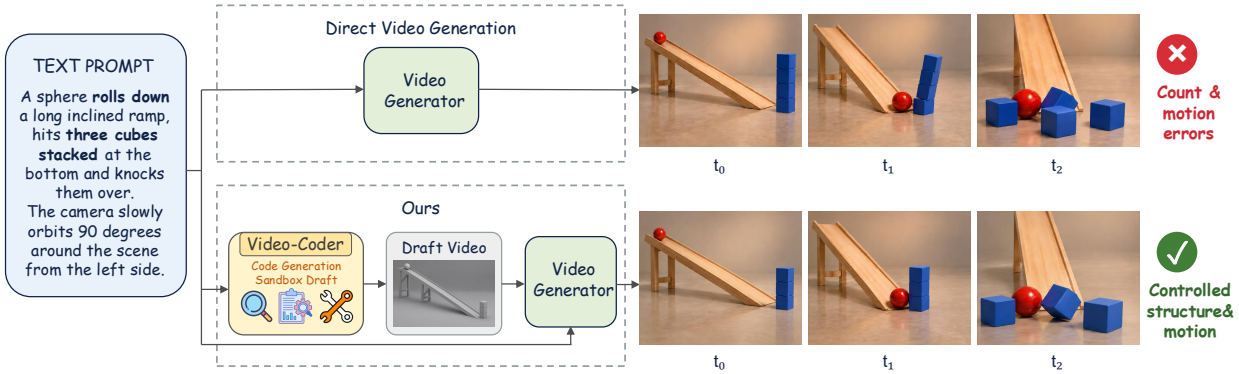


Figure 1: Overview of our proposed Video-Coder. Unlike direct video generation, Video-Coder writes and executes code to build a 3D scene and renders a draft video, which guides the downstream video generator with explicit structure and motion signals.

code to construct a dynamic 3D scene and renders it into an draft video, which explicitly specifies structure and motion and guides a downstream video generator to produce the final video. Since the scene is determined by an executable program, generation becomes controllable, interpretable, and verifiable. Through multi-turn interaction, the agent further observes execution errors and rendered results to refine its code.

To achieve this goal, we build a dedicated data construction pipeline. We collect diverse prompts from multiple video generation datasets and use Gemini 3.8 Flash [19] with the Video-Coder Harness to generate agentic trajectories, in which the model iteratively writes, executes, and revises code until the rendered Draft Video matches the prompt. The prompt and Draft Video are then fed into MiniMax-H3 [20] to synthesize a ground-truth reference video. After quality filtering, we obtain Video-Coder-SFT-6k and Video-Coder-RL-2k. In addition, we introduce the Video-Coder Harness, a unified interactive environment integrating code execution, 3D scene rendering, and execution feedback for agent training and inference.

Based on these resources, we train Video-Coder in two stages: SFT equips the model with fundamental scene-programming and tool-calling capabilities; while harness-native agentic RL further optimizes its programming and tool-calling trajectories. To ensure the executability of the generated program and the quality of the final video, we design two reward signals: execution-based reward and video-based Reward. The execution-based reward measures the executability of the generated program, while the video-based reward evaluates the quality of the generated video. Additionally, we introduce an annealing schedule that gradually reduces the impact of the execution-based reward throughout the training process, allowing the model to rely more on the video-based reward in later stages. In this way, execution-based reward stabilizes early-stage training, while the video-based reward gradually dominates in later stages and directly optimizes the final video quality. This design provides stable and informative learning signals for RL training, thereby improving both the reliability of code execution and the visual and motion quality of the generated videos.

Extensive experiments show that Video-Coder consistently improves video generation performance across different backbones, bringing around 7 points gains for MiniMax-H3 on PhyGenBench, demonstrating that explicitly constructing 3D scenes with executable code effectively enhances the physical plausibility and controllability of generated videos.

Our main contributions are summarized as follows:

- To the best of our knowledge, **Video-Coder** is the first attempt to explore training a multimodal coding agent for video generation. We hope that Video-Coder will serve as a foundation for future research on code-driven video generation.
- We design a dedicated data construction pipeline and curate two datasets, **Video-Coder-SFT-6k** and **Video-Coder-RL-2k**. We further introduce the **Video-Coder Harness**, a unified interactive environment integrating code execution, 3D scene rendering, and execution feedback for agent training and inference.
- Extensive experiments validate the effectiveness of our proposed Video-Coder. For example, our method improves Minimax-H3 by around 7 points on PhyGenBench and 24 points on VBench-2.0.

2 Related Works

2.1 Video Generation Models

Video generation has advanced rapidly with diffusion models [21, 22], particularly Diffusion Transformers [23] trained at scale. Representative systems include Sora [1], CogVideoX [24], HunyuanVideo [25], Wan [5], Kling [3], and Seedance [4], while autoregressive models such as VideoPoet [26] offer an alternative paradigm. Despite their visual fidelity, recent benchmarks [9, 6, 8, 7] show that these models frequently violate object counts, spatial relations, and physical laws, and Kang *et al*. [10] find that scaling alone is insufficient to learn such laws. To improve controllability, prior work conditions generation on explicit signals such as depth [27], trajectories and camera poses [28, 29], 3D tracking videos [30], or physical simulation [31, 32]. However, these signals are typically provided by users or produced by hand-crafted pipelines. Video-Coder acts as a coding agent to automatically construct these signals, representing them as an executable 3D scene and rendering them into a draft video, thereby providing structural and motion signal guidance for the downstream generation model.

2.2 Code-Driven Visual Generation

LLMs have shown strong capabilities in code generation and agentic tasks [11, 12, 13, 33], motivating their use as planners or programmers for visual content creation. Early works use LLMs to produce 2D layouts that guide diffusion models [34, 18, 17], which, however, cannot faithfully describe 3D geometry, occlusion, or camera motion. Others adopt executable programs as the 3D representation, generating or editing scenes via Blender [35] code [14, 15, 16]. Closer to our setting, GPT4Motion [32] scripts physical motions in Blender to guide video diffusion, and Video-CoCo [36] uses code as an intermediate chain-of-thought for physically consistent video generation. Nevertheless, most of these approaches rely on prompting frozen, often proprietary, models within fixed pipelines, without optimizing the model for the quality of the final video. In contrast, Video-Coder trains a multimodal coding agent to write, execute, and revise programs over multiple turns in an executable and verifiable harness.

2.3 Agentic Reinforcement Learning

Agentic reinforcement learning (RL) has recently emerged as an effective paradigm for training large language model (LLM) agents to perform multi-step reasoning and tool interaction [37, 38, 39]. Building on RL with verifiable rewards [40, 41, 42, 43], recent studies explore training agents that interact with external tools and environments through long-horizon trajectories. For example, Search-R1 [37] trains LLMs to interleave reasoning with real-time search engine calls, masking retrieved tokens to stabilize optimization. ReTool [38] integrates real-time code execution into the reasoning process, enabling the model to learn when and how to invoke a code interpreter from outcome rewards. SWE-RL [44] scales RL to real-world software engineering with lightweight rule-based rewards that measure the similarity between generated and ground-truth patches. ARPO [45] introduces an entropy-based adaptive rollout mechanism tailored to multi-turn tool-use agents. AdaTooler-V [46] proposes an adaptive tool-usage framework for image and video tool use, which dynamically adjusts reward scales to encourage invoking visual tools only when they provide measurable benefits. Gen-Searcher [47] further extends agentic RL to image generation, training an multimodal deep search agent for image generation. However, training coding agents for physically consistent and temporally coherent video generation using agentic reinforcement learning remains largely unexplored.

3 Method

3.1 Dataset Construction

High-quality training data is essential for training a coding agent for video generation. However, such data does not naturally exist: each sample must simultaneously contain (i) a text prompt describing a dynamic scene; (ii) a complete multi-turn agentic coding trajectory; and (iii) a grounded video. To address this challenge, we design a four-stage data construction pipeline (see Figure 2), consisting of text prompt construction, agentic trajectory generation, grounded video synthesis, and data cleaning and filtering.

Text Prompt Construction. We first collect candidate prompts from the captions of multiple public video generation training datasets [48, 49, 50], covering a wide range of domains, including everyday indoor and outdoor scenes,

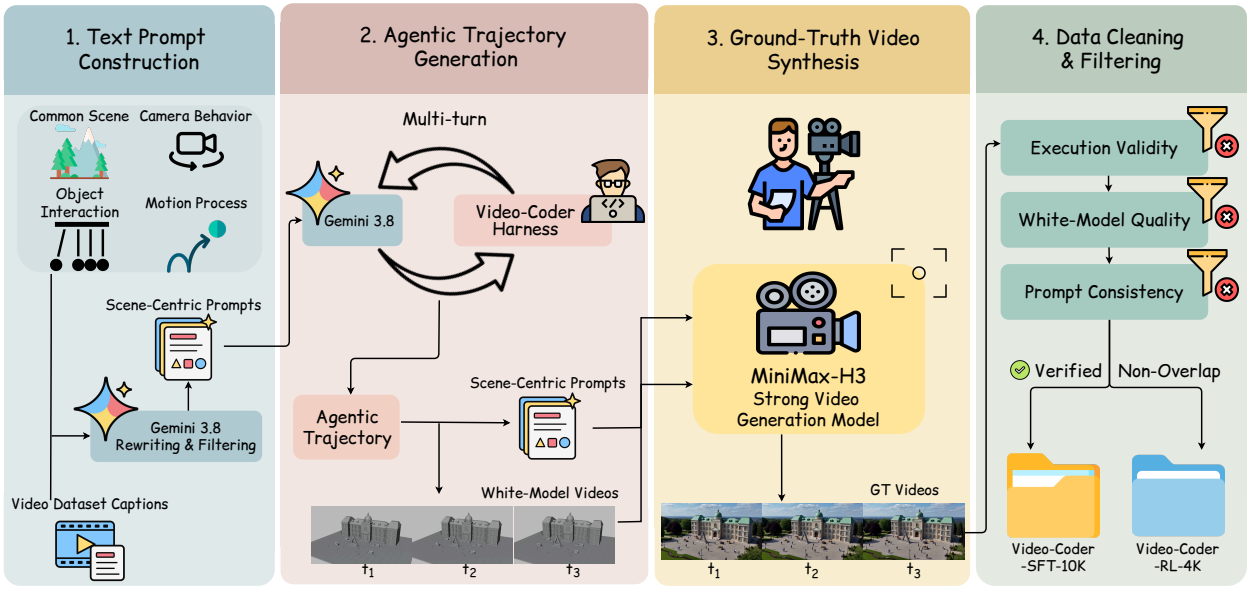


Figure 2: Overview of our data construction pipeline. It consists of four stages: text prompt construction, agentic trajectory generation, ground-truth video synthesis, and data cleaning and filtering, yielding two datasets: Video-Coder-SFT-6k and Video-Coder-RL-2k.

object interactions, physical phenomena (*e.g.*, falling, collision, rolling, and bouncing), and camera motion. Since these captions are written for pixel-level generation, they typically emphasize appearance attributes such as texture, lighting, and style, and often contain content that cannot be constructed from geometric primitives and keyframed motion alone. We therefore use an LLM to rewrite and filter them. On the one hand, each prompt is rewritten into a scene-centric description that explicitly specifies the objects involved and their counts, spatial layout, motion process, and camera behavior. On the other hand, we discard prompts that primarily rely on fine-grained appearance, human facial expressions, or other content that cannot be effectively modeled through 3D scene programming.

Agentic Trajectory Generation. Given the constructed text prompts, we generate agentic coding trajectories, in which the agent iteratively writes, executes, revises Blender code and render a Draft Video in the Video-Coder Harness. The resulting Draft Video provides precise structure and motion guidance for the subsequent ground-truth video synthesis. Meanwhile, these trajectories also serve as valuable supervision data for the subsequent supervised fine-tuning (SFT). Specifically, we connect Gemini 3.8 Flash [19] to the Video-Coder Harness (Section 3.2). For each prompt, the model plans the scene and writes Blender Python code to create objects, animate their motion, and configure the camera, then executes it in the Harness, which returns error messages on failure or rendered frames on success. Based on this feedback, the model revises its code and re-renders the scene until the Draft Video matches the prompt or the turn limit $T_{\max}$ is reached. We record each episode, including reasoning, code, tool calls, execution feedback, and rendered observations, as a multi-turn trajectory.

Ground-Truth Video Synthesis. The draft video determines the geometric structure, spatial layout, and motion of the scene, but contains no textures or realistic appearance. We feed the text prompt together with the draft video of each agentic trajectory into MiniMax-H3 [20], which synthesizes a video whose structure and motion follow the white-model video and whose appearance follows the prompt. The generated videos serve as synthesis ground truth for training the coding agent.

Data Cleaning and Filtering. The raw outputs of the above stages inevitably contain failed or low-quality samples. We therefore apply filtering at three levels: (1)Execution validity: we discard trajectories that never render successfully, exceed the turn or token budget, or contain invalid tool calls; (2)White-model video quality: we remove videos that are nearly static, empty, have the main subject out of frame, or exhibit degenerate camera viewpoints; (3)Prompt–video consistency: we use **Seed2.0** [51] to judge whether the white-model video correctly reflects the objects, counts, spatial relationships, and motion described in the prompt, and remove samples scoring below a threshold.

After filtering, we split the data into two non-overlapping subsets.**Video-Coder-SFT-6k** contains 6k samples, each consisting of a prompt, a complete agentic trajectory, and a ground-truth reference video, and is used for super-

vised fine-tuning.**Video-Coder-RL-2k** contains 2k prompt–reference video pairs and is used for agentic reinforcement learning.

3.2 Video-Coder Harness

Generating high-quality Blender programs typically requires iterative refinement, but reliance on human feedback makes this process difficult to scale. To this end, we build the **Video-Coder Harness**, an environment in which model can run its code, observe the outcome, and revise accordingly. It is used for agentic trajectory synthesis (Section 3.1), agentic RL (Section 3.3).

Task formulation. Given a text prompt c, the model π writes a Blender Python program f that builds a dynamic 3D scene. Executing f in the Harness yields

$$(v, \ell) = \mathcal{E}(f), \tag{1}$$

where v is the rendered white-model video (or $\varnothing$ if execution fails) , ℓ is the execution feedback and $\mathcal{E}$ is denotes the process of executing a program in sandbox and rendering the resulting scene into a video. Let $f^{(t)}$ be the program written at turn t., the agent iterates in a write–run–revise loop:

$$f^{(t)} = \pi\big(c, \{f^{(t)}, \ell^{(t)}, v^{(t)}\}_{t<T}\big), \tag{2}$$

where T denotes the final turn, and at each turn, the model refines the program based on previously written code and the visual feedback obtained.

Unified rendering setting. Each episode runs in an isolated sandbox with Blender installed. To keep the white-model videos consistent and focused on what matters for downstream generation, all scenes are rendered under one shared setting: untextured gray clay shading, a fixed resolution and frame rate. The resulting video thus carries only structure and motion. While appearance is left to the downstream video generator.

Tools and feedback. The agent edits its python script with ordinary shell commands and verifies it with two tools: `check_script`, a fast static check, and `run_blender`, which executes the script and renders the draft video. After each tool call, the Harness returns three types of feedback: (1)Execution feedback. Syntax errors, missing cameras or render calls, and runtime tracebacks. If execution fails, this is the only feedback. (2)Content checks. For a successful render, the Harness verifies the video format and inspects what the camera actually sees, reporting empty frames, static videos, or objects that leave the view, together with a brief scene summary. (3)Visual feedback. A few evenly spaced frames of the rendered video are returned as images, allowing model to inspect its own output and adjust layout, motion timing, or camera framing.

Verification and acceptance. An episode ends when the agent submits or when a limit is reached. A submission is not trusted directly: the Harness re-runs the final script from scratch in a clean directory and accepts the episode only if this independent re-execution again passes all checks. This guarantees that every accepted result is reproducible from the code alone, and the re-rendered video $v^{(T)}$ is exported as the final white-model video.

3.3 Training Recipe

In this section, we train **Video-Coder** to act as a multimodal coding agent that iteratively writes, executes, and revises Blender programs for video generation. Our training recipe follows a two-stage pipeline, consisting of SFT and harness-native agentic RL.

Two-Stage Training. We initialize Video-Coder from **Qwen3.5-9B** [52]. In the first stage, we perform supervised fine-tuning on **Video-Coder-SFT-6k**. This stage teaches the model to act as a multi-turn coding agent: planning a dynamic 3D scene, writing Blender Python code, invoking the tools (`check_script` and `run_blender`), interpreting the returned execution and visual feedback, and revising the code accordingly. In the second stage, we further optimize the model on **Video-Coder-RL-2k** with harness-native agentic reinforcement learning, which enables the model to learn more effective scene-programming and revision strategies. It is worth noting that during training, the video generator remains fixed; we only optimize Qwen3.5-9B to produce executable Blender programs whose rendered white-model videos serve as structural and motion guidance for the final video generation.

Reward Feedback Design. To ensure both the executability of generated programs and the quality of final video generated by video generator, we design two reward signals: execution-based reward and video-based reward. The

execution reward R_{exe} measures the executability of generated programs. A reward of 1 is assigned if all tool call formats are correct and the final Blender program executes successfully and passes all environment checks; otherwise, the reward is 0. The video reward R_{video} employs **Qwen3.8-omni-flash** [53] to evaluate the consistency of the final generated video with the text prompt and reference video, focusing on object categories, spatial layout, motion dynamics, and camera behavior. The video reward is normalized to the range [0, 1]. The corresponding prompt can be found in Appendix B.

However, these two rewards play different roles at different stages of training. In the early training stages, the model frequently generates programs with execution errors, making video rewards difficult to obtain. Therefore, execution rewards are crucial for learning reliable tool calls and code generation. As training progresses and the code execution success rate improves, video rewards provide more fine-grained supervision, guiding the agent to generate videos that better align with the target requirements.

To dynamically balance these two reward signals, we introduce a time-based annealing strategy that gradually shifts the optimization focus from code execution correctness to video generation quality. Specifically, the final reward is defined as:

$$R = \alpha \cdot R_{video} + e^{-\frac{\texttt{steps}}{T}} \cdot R_{exe}, \tag{3}$$

where α is a balancing hyperparameter, `steps` denotes the current global training step and T is the total number of training steps. Here we simply set $\alpha = 0.5$.

Optimization. After the computing the final reward, we optimize the policy with GRPO [40]. For each prompt q, we sample G trajectories $\{o_i\}_{i=1}^{G}$ from $\pi_{\theta_{\text{old}}}$ and compute group-normalized advantages:

$$A_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^{G})}{\text{std}(\{R_j\}_{j=1}^{G})}. \tag{4}$$

The final policy update follows the standard GRPO objective:

$$\mathbb{E}_{q,\{o_i\}}\left[\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}, 1-\epsilon, 1+\epsilon\right)A_i\right) - \beta_{\text{KL}}\, D_{\text{KL}}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)\right)\right]. \tag{5}$$

where the definition of variables and hyperparameters follows the standard GRPO algorithm [40].

4 Experiment

4.1 Setup

Training Details. We train Video-Coder-9B using 8 NVIDIA H100 GPUs under our proposed Video-Coder Harness, with Qwen3.5-9B [52] as the base model. We first perform supervised fine-tuning on Video-Coder-SFT-10k, and then further conduct agentic RL training on Video-Coder-RL-4k. For both SFT and RL, we use AdamW [62] as the optimizer. The learning rate is set to 1×10^{-5} for SFT and 1×10^{-6} for RL, and the batch size is set to 8 in both stages. During RL training, we additionally deploy Minimax-H3 [20] on 16 H100 GPUs to support rollout Video generation. In addition, we set the group size to 8, limit the maximum number of interaction turns to 100 and set the maximum context length to 120K. Following prior practice [63], we mask out unfinished rollouts. Overall, the complete training process takes approximately 3 days.

Benchmarks and Evaluation We evaluated on two physics-focused video benchmarks. PhyGenBench [6] evaluates physical commonsense across four categories: mechanics, optics, thermal, and material dynamics. We follow its official evaluation protocol, which uses GPT-4o [64] as an MLLM judge to assess how faithfully a generated video adheres to the physical principles implied by its prompt, providing a comprehensive measure of the physical knowledge instantiated by a generative model. VBench-2.0 [7] aims to evaluate intrinsic faithfulness beyond surface quality; we focus on its physical dimensions (mechanics, thermotics, and material), with each dimension probed by a dedicated prompt suite and scored to reflect adherence to real-world physical laws. PhyGenBench reports a per-category consistency score in [0, 1] and their average. VBench-2.0 reports per-dimension plausibility as a percentage. Higher is better on every metric.

Table 1: Performance of different models on the PhyGenBench benchmark. Best performance is in **bold**.

Method	Mechanics (↑)	Optics (↑)	Thermal (↑)	Material (↑)	Average (↑)
Pika [54]	0.35	0.56	0.43	0.39	0.44
Gen-3 [55]	0.45	0.57	0.49	0.51	0.51
Kling [3]	0.45	0.58	0.50	0.40	0.49
CogVideoX [24]	0.39	0.55	0.40	0.42	0.45
Open-Sora V1.2 [56]	0.43	0.50	0.44	0.37	0.44
LaVie [57]	0.30	0.44	0.38	0.32	0.36
Vchitect-2.0 [58]	0.41	0.56	0.44	0.37	0.45
Minimax-H3 [20]	0.61	0.73	0.52	0.55	0.61
HunyuanVideo [25]	0.33	0.39	0.26	0.30	0.33
OmniWeaving [59]	0.48	0.56	0.43	0.39	0.48
Wan2.2-TI2V-5B [5]	0.55	0.58	0.53	0.50	0.54
Cosmos-Predict2.5 [60]	0.30	0.33	0.41	0.39	0.35
LTX-Video-2B [61]	0.51	0.58	0.48	0.45	0.51
Video-CoCo [36]	0.51	0.58	0.48	0.45	0.51
Video-Coder-9B + OmniWeaving	0.52	0.58	0.52	0.44	0.51
Video-Coder-9B + Minimax-H3	**0.68**	**0.79**	**0.60**	**0.61**	**0.67**

Table 2: Performance of different models on the VBench-2.0 benchmark. Best performance is in **bold**.

Method	Mechanics (↑)	Thermotics (↑)	Material (↑)	Average (↑)
Sora [1]	62.22%	43.36%	64.94%	56.84%
Kling 1.6 [3]	65.55%	59.46%	68.00%	64.34%
HunyuanVideo [25]	76.09%	56.52%	64.37%	65.66%
Minimax-H3 [20]	67.59%	47.22%	48.94%	54.58%
CogVideoX-1.5 [24]	80.80%	67.13%	**83.19**%	77.04%
Video-Coder-9B + Minimax-H3	**88.67**%	**70.88**%	75.02%	**78.19**%

4.2 Main Results

Results on PhyGenBench. As shown in Table 1, Video-Coder-9B + Mimax-H3 achieves the best average score of 0.67 among all compared methods, outperforming the strong open-source model Wan2.2-TI2V-5B (0.54), the HunyuanVideo (0.33), and commercial systems such as Gen-3 (0.51) and Kling (0.49). It ranks first on the all four categories, namely Mechanics (0.68), Optics (0.79), Thermal (0.60) and Material (0.61). Notably, although Video-Coder was trained with Minimax-H3 as the rollout generator during reinforcement learning (RL), it generalizes well to other generators at test time. Specifically, it improves OmniWeaving from 0.48 to 0.51 (an increase of 16.28 points). These results not only demonstrate the effectiveness of our coding agent, but also its strong transferability and robustness across video generators with vastly different native capabilities. Overall, these results indicate that our proposed Video-Coder is a versatile and powerful model capable of enhancing the physical consistency of video generation.

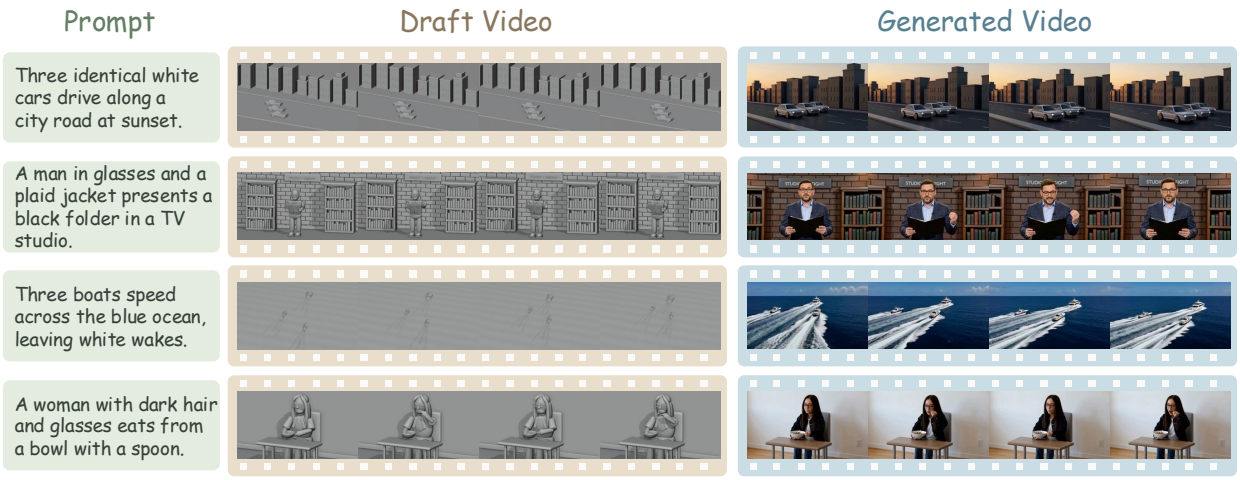


Figure 3: Qualitative examples of Video-Coder. Each row shows a text prompt, sampled frames from the rendered white-model draft video, and the corresponding generated video. The four cases illustrate diverse scenes involving vehicles, human actions, and object interactions.

Results on VBench-2.0. Table 2 reports results on the physical dimensions of VBench-2.0. Video-Coder-9B + MiniMax-H3 achieves the highest average plausibility of 78.19%, surpassing all baselines including CogVideoX-1.5 (77.04%), and improves over MiniMax-H3 by 24 points. Especially on mechanics, our method reaches 88.67%, exceeding the best baseline CogVideoX-1.5 (80.80%) by 7.87 points and its base generator MiniMax-H3 (67.59%) by 21.08 points. This dimension directly probes motions governed by mechanical laws, such as falling, collision, and rolling, which Video-Coder explicitly specifies through keyframed or physics-simulated motion in the Blender program. Our method also achieves the best Thermotics score (70.88%). On Material, Video-Coder improves over MiniMax-H3 (75.02% *vs*. 48.94%) but still lags behind CogVideoX-1.5 (83.19%). We attribute this to the fact that material-related phenomena depend heavily on fine-grained appearance and material response, whereas the white-model video intentionally encodes only structure and motion.

Qualitative Results. Figure 3 presents qualitative examples covering vehicles, human actions, and object interactions. The white-model drafts produced by Video-Coder explicitly determine object counts (*e.g*., three cars and three boats), spatial layout, motion direction, and camera framing, and the generated videos faithfully preserve these properties while adding realistic appearance as described in the prompts. These examples illustrate that decoupling structure and motion from appearance enables precise control over scene composition that is difficult to achieve through prompting alone.

4.3 Ablation Study

To verify the contribution of each component in Video-Coder, we conduct ablation studies on PhyGenBench. All variants use MiniMax-H3 [20] as the video generator, and we report the average score. Specifically, we compare the following variants: (1) *MiniMax-H3*, the vanilla generator that directly synthesizes videos from the original prompts; (2) + *Video-Coder-SFT*, which trains Video-Coder with SFT only; (3) + *Video-Coder-wo-annealing*, which combines the two rewards with fixed weights, *i.e*., $R = \alpha \cdot R_{\text{video}} + R_{\text{exe}}$; (4) + *Video-Coder-wo-video reward*, which uses only the execution reward during RL; (5) + *Video-Coder-wo-execution reward*, which uses only the video reward during RL; and (6) + *Video-Coder*, the full model trained with SFT followed by agentic RL. The results are shown in Table 3.

Compared with the plain Minimax-H3 baseline, Video-Coder-SFT improves the score from 0.61 to 0.63, confirming that explicitly constructing the 3D scene through code provides useful structure and motion priors for physically grounded generation. By imitating complete agentic trajectories, the model acquires the fundamental abilities of scene programming, tool use, and feedback-driven code revision. Agentic RL further improves the score to 0.67, bringing an overall improvement of 0.06 over the vanilla generator. This shows that while SFT provides a strong initialization for the model's scene modeling capabilities, RL remains essential for further optimizing multi-turn scene programming behaviors and generating draft videos that more faithfully adhere to the input prompts, ultimately leading

Table 3: Ablation study of Video-Coder on PhyGenBench.

Methods	PhyGenBench
Minimax-H3	0.61
+ Video-Coder-SFT	0.63
+ Video-Coder-wo-annealing	0.65
+ Video-Coder-wo-video reward	0.64
+ Video-Coder-wo-execution reward	0.63
+ Video-Coder	**0.67**

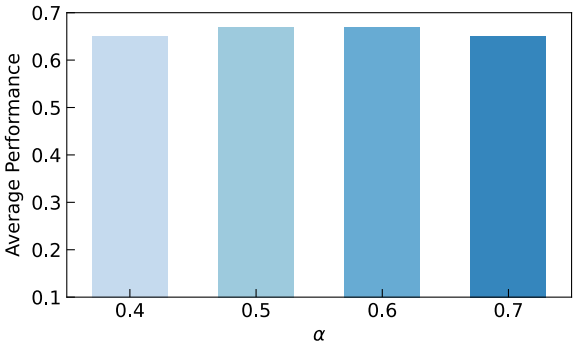


Figure 4: Sensitivity analysis of α on PhyGenBench.

to higher-quality final videos. Moreover, removing either the video reward or the execution reward leads to clear performance degradation, with scores dropping to 0.64 and 0.63, respectively. This confirms that the two reward signals play complementary roles. Specifically, the execution reward supervises the model to achieve stable early-stage training, while the video reward provides fine-grained supervision for structure, motion, and camera behavior. Finally, Video-Coder-wo-annealing yields a score of only 0.65. The performance drop may be due to the over-exploitation of the execution reward: it masks the fine-grained differences in the video reward, encouraging the agent to generate executable yet conservative programs. The annealing strategy encourages the model to rely more on video quality-based rewards in the later training stages , alleviating this issue.

4.4 Parameter Sensitivity Analysis

We further conducted a sensitivity analysis on the magnitude of the video reward controlled by the hyperparameter α. Fig. 4 demonstrates the performance of our Video-Coder using different α for RL training on PhyGenBench. We observe that performance drops slightly at $\alpha = 0.4$ and $\alpha = 0.7$, while $\alpha = 0.5$ and $\alpha = 0.6$ yield similar and favorable results. A small α provides insufficient supervision on video quality, whereas a large α weakens the execution signal that stabilizes early training. This indicates that within a reasonable range, the model is relatively insensitive to the choice of α.

5 Conclusion

In this paper, we present Video-Coder, the first attempt to train a multimodal coding agent for physically consistent video generation. To support this paradigm, we develop a dedicated data pipeline, construct two training datasets Video-Coder-SFT-10k and Video-Coder-RL-4k. Moreover, we introduce the Video-Coder Harness, a unified interactive environment integrating code execution, 3D scene rendering, and execution feedback for agent training and inference. Based on these resources, we train Video-Coder through a two-stage scheme of SFT and agentic RL. Experiments on PhyGenBench and VBench-2.0 show that Video-Coder substantially improves the physical consistency of generated videos and generalizes across different video generators. We hope this work serves as an open foundation for controllable, interpretable, and physically consistent video generation.

References

[1] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. https://openai.com/research/video-generation-models-as-world-simulators, 2024. OpenAI Technical Report.

[2] Google DeepMind. Veo 3. https://deepmind.google/models/veo/, 2025.

[3] Kuaishou. Kling ai. https://klingai.com, 2024.

[4] Yu Gao et al. Seedance 1.0: Exploring the boundaries of video generation models. *arXiv preprint arXiv:2506.09113*, 2025.

[5] Team Wan et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[6] Fanqing Meng et al. Towards world simulator: Crafting physical commonsense-based benchmark for video generation. In *ICML*, 2025.

[7] Dian Zheng et al. Vbench-2.0: Advancing video generation benchmark suite for intrinsic faithfulness. *arXiv preprint arXiv:2503.21755*, 2025.

[8] Saman Motamed, Laura Culp, Kevin Swersky, Priyank Jaini, and Robert Geirhos. Do generative video models understand physical principles? *arXiv preprint arXiv:2501.09038*, 2025.

[9] Hritik Bansal et al. Videophy: Evaluating physical commonsense for video generation. In *ICLR*, 2025.

[10] Bingyi Kang et al. How far is video generation from world model: A physical law perspective. In *ICML*, 2025.

[11] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[12] John Yang et al. Swe-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

[13] Xingyao Wang et al. Openhands: An open platform for ai software developers as generalist agents. In *ICLR*, 2025.

[14] Chunyi Sun et al. 3d-gpt: Procedural 3d modeling with large language models. *arXiv preprint arXiv:2310.12945*, 2023.

[15] Ziniu Hu et al. Scenecraft: An llm agent for synthesizing 3d scenes as blender code. In *ICML*, 2024.

[16] Ian Huang et al. Blenderalchemy: Editing 3d graphics with vision-language models. In *ECCV*, 2024.

[17] Han Lin et al. Videodirectorgpt: Consistent multi-scene video generation via llm-guided planning. In *COLM*, 2024.

[18] Long Lian et al. Llm-grounded video diffusion models. In *ICLR*, 2024.

[19] Google DeepMind. Gemini 3.8 flash, September 2026.

[20] MiniMax. Hailuo ai. https://hailuoai.video, 2024.

[21] Jonathan Ho et al. Video diffusion models. In *NeurIPS*, 2022.

[22] Andreas Blattmann et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023.

[23] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *ICCV*, 2023.

[24] Zhuoyi Yang et al. Cogvideox: Text-to-video diffusion models with an expert transformer. In *ICLR*, 2025.

[25] Weijie Kong et al. Hunyuanvideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024.

[26] Dan Kondratyuk et al. Videopoet: A large language model for zero-shot video generation. In *ICML*, 2024.

[27] Patrick Esser et al. Structure and content-guided video synthesis with diffusion models. In *ICCV*, 2023.

[28] Zhouxia Wang et al. Motionctrl: A unified and flexible motion controller for video generation. In *SIGGRAPH*, 2024.

[29] Hao He et al. Cameractrl: Enabling camera control for text-to-video generation. In *ICLR*, 2025.

[30] Zekai Gu et al. Diffusion as shader: 3d-aware video diffusion for versatile video generation control. In *SIGGRAPH*, 2025.

[31] Shaowei Liu et al. Physgen: Rigid-body physics-grounded image-to-video generation. In *ECCV*, 2024.

[32] Jiaxi Lv et al. Gpt4motion: Scripting physical motions in text-to-video generation via blender-oriented gpt planning. In *CVPR Workshops*, 2024.

[33] Dongyang Chen, Chaoyang Wang, Dezhao Su, Xi Xiao, Zeyu Zhang, Jing Xiong, Qing Li, Yuzhang Shang, and Shichao Kan. V-retrver: Evidence-driven agentic reasoning for universal multimodal retrieval. *arXiv preprint arXiv:2602.06034*, 2026.

[34] Weixi Feng et al. Layoutgpt: Compositional visual planning and generation with large language models. In *NeurIPS*, 2023.

[35] Blender Online Community. Blender – a 3d modelling and rendering package. https://www.blender.org.

[36] Haodong Li, Tianfei Ren, Xiaoxiao Ma, Chunmei Qing, Zhen Fang, Sipeng He, Ziyu Guo, Haoyu Wu, Juanxi Tian, Yihang Zou, et al. Videococo: Code-as-cot for physically-consistent video generation via an agentic dual-engine system. *arXiv preprint arXiv:2607.27380*, 2026.

[37] Bowen Jin et al. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

[38] Jiazhan Feng et al. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025.

[39] Zihan Wang et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025.

[40] Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[41] Daya Guo et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[42] Qiying Yu et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[43] Chaoyang Wang, Yangfan He, Yiyang Zhou, Yixuan Wang, Jiaqi Liu, Peng Xia, Zhengzhong Tu, Mohit Bansal, and Huaxiu Yao. Knowing the answer isn't enough: Fixing reasoning path failures in lvlms. *arXiv preprint arXiv:2512.06258*, 2025.

[44] Yuxiang Wei et al. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.

[45] Guanting Dong et al. Agentic reinforced policy optimization. *arXiv preprint arXiv:2507.19849*, 2025.

[46] Chaoyang Wang, Kaituo Feng, Dongyang Chen, Zhongyu Wang, Zhixun Li, Sicheng Gao, Meng Meng, Xu Zhou, Manyuan Zhang, Yuzhang Shang, et al. Adatooler-v: Adaptive tool-use for images and videos. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 18058–18072, 2026.

[47] Kaituo Feng, Manyuan Zhang, Shuang Chen, Yunlong Lin, Kaixuan Fan, Yilei Jiang, Hongyu Li, Dian Zheng, Chenyang Wang, and Xiangyu Yue. Gen-searcher: Reinforcing agentic search for image generation. *arXiv preprint arXiv:2603.28767*, 2026.

[48] Yi Wang, Yinan He, Yizhuo Li, Kunchang Li, Jiashuo Yu, Xin Ma, Xinyuan Chen, Yaohui Wang, Ping Luo, Ziwei Liu, Yali Wang, Limin Wang, and Yu Qiao. Internvid: A large-scale video-text dataset for multimodal understanding and generation. *arXiv preprint arXiv:2307.06942*, 2023.

[49] Kepan Nan, Rui Xie, Penghao Zhou, Tiehan Fan, Zhenheng Yang, Zhijie Chen, Xiang Li, Jian Yang, and Ying Tai. Openvid-1m: A large-scale high-quality dataset for text-to-video generation. *arXiv preprint arXiv:2407.02371*, 2024.

[50] Xuan Ju, Yiming Gao, Zhaoyang Zhang, Ziyang Yuan, Xintao Wang, Ailing Zeng, Yu Xiong, Qiang Xu, and Ying Shan. Miradata: A large-scale video dataset with long durations and structured captions, 2024.

[51] Bytedance Seed. Seed2. 0 model card: Towards intelligence frontier for real-world complexity. *arXiv preprint arXiv:2607.00248*, 2026.

[52] Qwen Team. Qwen3.5. https://github.com/QwenLM, 2026.

[53] Qwen Team. Qwen3. 8-omni: Towards native omni-modal agents. *arXiv preprint arXiv:2609.25611*, 2026.

[54] Pika Labs. Pika. https://pika.art, 2023.

[55] Runway. Introducing gen-3 alpha: A new frontier for video generation. https://runwayml.com/research/introducing-gen-3-alpha, 2024.

[56] Zangwei Zheng et al. Open-sora: Democratizing efficient video production for all. *arXiv preprint arXiv:2412.20404*, 2024.

[57] Yaohui Wang et al. Lavie: High-quality video generation with cascaded latent diffusion models. *arXiv preprint arXiv:2309.15103*, 2023.

[58] Weichen Fan et al. Vchitect-2.0: Parallel transformer for scaling up video diffusion models. *arXiv preprint arXiv:2501.08453*, 2025.

[59] Kaihang Pan, Qi Tian, Jianwei Zhang, Weijie Kong, Jiangfeng Xiong, Yanxin Long, Shixue Zhang, Haiyi Qiu, Tan Wang, Zheqi Lv, et al. Omniweaving: Towards unified video generation with free-form composition and reasoning. *arXiv preprint arXiv:2603.24458*, 2026.

[60] NVIDIA. World simulation with video foundation models for physical ai. *arXiv preprint arXiv:2511.00062*, 2025.

[61] Yoav HaCohen et al. Ltx-video: Realtime video latent diffusion. *arXiv preprint arXiv:2501.00103*, 2024.

[62] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[63] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *Advances in Neural Information Processing Systems*, 38:113222–113244, 2026.

[64] Aaron Hurst et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.

A System Prompt of Video-Coder

Figures 5–7 present the complete system prompt used by Video-Coder. The same prompt is used for agentic trajectory generation with the teacher agent, agentic RL training, and inference.

System Prompt (1/3)

```
## Task

Write a Blender {{ blender_version }} Python script that builds the following scene and renders it as an animated white-model
video:

<scene_description>
{{ task }}
</scene_description>


## Script contract

- Write exactly one self-contained script: `{{ script_name }}` in the current directory.
- It is executed headless as `blender --background --factory-startup --python {{ script_name }}` with Blender {{
blender_version }}, from the directory the video must be written to. Use only the Blender {{ blender_version }} API and the
Python standard library; no external assets, files or add-ons.
- Start by removing every object in the default scene, then build the scene from scratch.
- Geometry: model with primitives, modifiers, `bmesh` or `mesh.from_pydata`. Name objects meaningfully. Keep the total scene
under ~500k faces. Objects must not intersect unless the description implies contact.
{% if max_frame_count > frame_count -%}
- Animation: the video lasts {{ (frame_count / fps) | round | int }}-{{ (max_frame_count / fps) | round | int }} seconds at
{{ fps }} fps, i.e. {{ frame_count }}-{{ max_frame_count }} frames; choose the length that suits the action. Set
`scene.frame_start = 1` and `scene.frame_end` to that length, and animate with keyframes, drivers, constraints or physics so
that the motion described actually happens within that range. Bake any physics simulation before rendering.
{%- else -%}
- Animation: the video has {{ frame_count }} frames at {{ fps }} fps. Set `scene.frame_start = 1` and `scene.frame_end = {{
frame_count }}`, and animate with keyframes, drivers, constraints or physics so that the motion described actually happens
within that range. Bake any physics simulation before rendering.
{%- endif %}
- Camera: create a camera, assign it to `scene.camera`, and frame the action so that it stays in view for all frames (animate
the camera if the description asks for camera motion).
- Seed any randomness (`random.seed(...)`) so the result is deterministic. Prefer `bpy.data` / low-level API over `bpy.ops`
where the operator needs UI context.
- Do not import subprocess, socket, shutil, urllib or similar modules, and do not call os.system/eval/exec.

## White-model render (required, at the end of the script)

Every video in this dataset uses the same look, so configure exactly this and then render:

- `scene.render.engine = 'BLENDER_WORKBENCH'`
- `scene.display.shading`: `light = 'STUDIO'`, `color_type = 'SINGLE'`, `single_color = (0.8, 0.8, 0.8)`, `show_shadows =
True`, `show_cavity = True`, `cavity_type = 'WORLD'`. No materials or textures are needed; Workbench studio lighting ignores
scene lights.
- Resolution {{ resolution[0] }}x{{ resolution[1] }} at `resolution_percentage = 100`, `scene.render.fps = {{ fps }}`.
- Output: `scene.render.image_settings.file_format = 'FFMPEG'`, `scene.render.ffmpeg.format = 'MPEG4'`,
`scene.render.ffmpeg.codec = 'H264'`, `scene.render.filepath = os.path.join(os.getcwd(), '{{ output_video }}')`.
- Last statement: `bpy.ops.render.render(animation=True)`.
- Write no other files: no .blend files, images or caches outside the render. Do not quit Blender.
```

Figure 5: System prompt of Video-Coder (part 1 of 3).

System Prompt (2/3)

Tools

{% if blender_available -%}
Blender {{ blender_version }} is available through two commands (do not install anything):

```bash
check_script {{ script_name }} # fast static check: syntax, bpy import, camera, render call, forbidden calls
run_blender {{ script_name }} # executes the script exactly like the harness and renders {{ output_video }} (CPU, ~20-120 s)
```

`run_blender` reports the Python traceback if the script fails, whether {{ output_video }} has {% if max_frame_count > frame_count %}{{ frame_count }}-{{ max_frame_count }} frames matching the scene frame range{% else %}{{ frame_count }} frames{% endif %} at {{ resolution[0] }}x{{ resolution[1] }}, and a scene summary: object counts, animated objects, world positions of moving meshes at sampled frames, and meshes whose origin is outside the camera view. Use the numbers to verify that the motion, timing and framing match the description.{% if visual_feedback %} It also returns {{ preview_frames }} frames sampled evenly from the rendered video as images, with their frame numbers and times.{% endif %} It reports a problem when most frames are empty, nothing moves, or no object is in view.
{%- else -%}
Blender is NOT installed in this environment and must not be installed (no `pip install bpy`, no downloads). You cannot execute the script, so reason carefully about the Blender {{ blender_version }} API: exact operator names, property names and argument names.
The only verification tool is the static checker:

```bash
check_script {{ script_name }}
```

It checks syntax, required structure (bpy import, camera, render call) and forbidden calls. Passing it does not prove the script runs, so review the code yourself before submitting.
{%- endif %}

Workflow

1. Think & Plan: Before generating any code, explicitly reason about the scene: objects, their dimensions and positions in meters, the motion over time (frame timing), and camera placement/framing.
2. Write `{{ script_name }}` with a heredoc: `cat <<'EOF' > {{ script_name }}` ... `EOF`.
3. Run `check_script {{ script_name }}` and fix every error.
{% if blender_available -%}
4. Run `run_blender {{ script_name }}`. Fix every error and problem it reports, and compare the scene summary against the description (positions, timing, clearances, camera framing).{% if visual_feedback %}
Then inspect the returned frames one by one against the description and write down every concrete problem you see: missing, unrecognizable or wrongly counted objects; wrong arrangement; motion that is missing, in the wrong order or at the wrong time; objects cut off, outside the frame or too small; objects passing through each other; wrong camera behavior. Fix the problems in the script and run `run_blender` again.{% endif %}
Repeat until it reports RUN OK{% if visual_feedback %} and the frames clearly show every element of the description{% else %} and the numbers match{% endif %}. If a command keeps giving the same result, change the script or your approach instead of repeating it; repeated identical commands end the task.

Figure 6: System prompt of Video-Coder (part 2 of 3).

System Prompt (3/3)

Workflow (continued)

5. Submit with `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT` as a command on its own. You cannot continue after that. The harness then re-executes `{{ script_name }}` from scratch; the task only counts if it renders a valid video{% if judge_enabled %} and a reviewer who compares frames of the video against the description confirms that every element it describes (objects, arrangement, motion, camera) is clearly shown{% endif %}.
{%- else -%}
4. Re-read the script (`nl -ba {{ script_name }}`) and check API usage, coordinates, keyframe timing and camera framing against the description; fix problems with a rewrite or `sed -i`.
5. When the script is final and passes the check, submit with `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT` as a command on its own. You cannot continue after that.
{%- endif %}

Command rules

- Each command runs in a fresh subshell in the task directory; `cd` and environment variables do not persist.
- Only create or modify files in the current directory.
- Use only `check_script` and `run_blender` to check and execute scripts. Do not inspect or modify the harness itself (how these commands are implemented, `VCSFT_*` environment variables, the Blender/bpy installation) and do not run Blender or bpy any other way; such commands are blocked.
- For a quick experiment, write a small script in the current directory and run `run_blender experiment.py`: whatever it `print()`s appears in the blender log. Delete experiment files before submitting.

Figure 7: System prompt of Video-Coder (part 3 of 3).

B Video Reward Prompt

Figures 8 and 9 present the complete prompt used to compute the video reward R_{video} during agentic RL. Given the text prompt, the reference video, and the generated video, the VLM judge evaluates their consistency in terms of object categories, spatial layout, motion dynamics, and camera behavior, and the resulting score is normalized to $[0, 1]$.

Video Reward Prompt (1/2)

You are a strict, professional evaluator of AI-generated videos that reconstruct an animated scene.

A video model turns an untextured 3D animation (a "white model") and a scene description into a realistic video.
You will receive:
1) The scene description, at the end of this message.
2) Video 1: the reference video, made by the video model from a ground-truth animation of the description.
3) Video 2: the candidate video, made by the same video model with the same description and settings, from an animation written by the system under evaluation (the one being evaluated).
All people and objects in both videos are AI-generated.

Critical clarifications:
- This is NOT a pixel-level similarity task. The video model invents the appearance on its own, so it differs even between two perfect animations. Judge what the underlying animation determines: which objects are present and recognizable, where they are, how they move, and how the camera frames them.
- Ignore appearance: textures, colors, materials, lighting, background style, rendering style (stylized versus photorealistic), faces and clothing details, audio and image quality. Requirements of that kind in the description are not constraints. Do not reward the candidate for looking nicer than the reference.
- Do NOT assume correctness. A required element that is out of frame, hidden, too small, cut off by the frame or not recognizable counts as missing. A static video where the description requires action must score low on motion.
- Both videos have the same length: compare timing directly, ignoring offsets under about half a second.
- Video 1 is a reference, not the only valid answer: use it to judge layout, motion and camera, and use the description to judge faithfulness.

Required first step (it must appear in the rationale): extract 3-6 hard constraints from the description that concern visible content: required subjects/objects and their counts, key setting elements, required actions or motions (with direction or order if given), spatial relations, and camera requirements if the description states any.

Score Video 2 on four dimensions. Each score MUST be exactly one of 0, 0.25, 0.5, 0.75, 1. A score of 1 is rare and requires full success in that dimension; when torn between two levels, choose the lower one.

1) faithfulness: Video 2 against the description (presence and structure only; it need not match Video 1).
1: every hard constraint is clearly visible: all required subjects/objects with the right counts, the setting, and the required actions and relations.
0.75: all essential elements are present; one secondary element is missing, simplified or slightly off (for example the count of a minor prop).
0.5: the main subject and setting are there, but one important element is missing or wrong (wrong number of main subjects, a required action not performed, a key prop absent).
0.25: only a few elements of the description can be recognized; the scene is largely not what it asks for.
0: nothing in Video 2 recognizably depicts the description.

2) layout: Video 2 against Video 1, the spatial arrangement of the scene.
1: the same main objects in the same arrangement: relative positions, facing directions, relative sizes and placement in the setting all match.
0.75: the same arrangement with small differences in spacing, orientation or scale.
0.5: recognizably the same arrangement, with one clear difference (a main object in a different place, a clearly wrong relative size, a missing or extra main object).
0.25: only partial correspondence (for example a similar setting but a different arrangement of the main subjects).
0: an unrelated arrangement, or nothing comparable.

Figure 8: Prompt for the VLM-based video reward (part 1 of 2).

Video Reward Prompt (2/2)

3) motion: Video 2 against Video 1, what moves and how.
1: the same objects move along the same directions and paths, in the same order, with matching timing.
0.75: the same motions in the same order, with small differences in path, amplitude, speed or timing.
0.5: the main motion is present, but one important motion is missing, extra or in the wrong direction, or the order or timing clearly differs.
0.25: only a small part of the motion corresponds, or Video 2 is mostly static while Video 1 shows the main action.
0: no corresponding motion.
Set "motion_na": true only if the description requires no action and Video 1 shows no meaningful object motion; then output "motion": 0.5 (it is not used). Otherwise "motion_na": false.

4) camera: Video 2 against Video 1, viewpoint, framing and camera movement.
1: the same viewpoint (side, height, angle), the same shot size and framing of the main subjects, and the same camera movement (or both static).
0.75: the same viewpoint and movement, with small differences in distance, angle or framing.
0.5: one clear difference (a different shot size, a different viewing angle, or static versus moving) while the main subjects are still framed comparably.
0.25: several differences; the main subjects are framed very differently (for example a close-up versus a wide shot).
0: the camera does not show the main subjects.

Global rule: if Video 2 is empty, almost entirely blank, broken, or shows nothing recognizable for most of its duration, every score is 0 and "motion_na" is false.

Rationale: 5-10 short sentences, evidence only (describe what is visible, with approximate times where useful).
- Start with "Constraints:" and list the extracted hard constraints.
- Then give the key comparisons: how Video 2 meets or misses each constraint, and how its layout, motion and camera compare with Video 1.

Output ONLY one valid JSON object with exactly these keys, in this order, and nothing else (no markdown):
{"rationale": "<...>", "faithfulness": <score>, "layout": <score>, "motion": <score>, "camera": <score>, "motion_na": <true|false>}

Scene description:
{{scene_description}}

Figure 9: Prompt for the VLM-based video reward (part 2 of 2).